Restaurant Review Platform (Web and Mobile)

Solution write up

Team

Krishantha Dinesh – Architect and Backend Development

Denuwan Bandara – Senior Engieer Integration and web

Dilshika Bandara – Web UI and mobile development

1. Introduction

This report presents the design and implementation of a cross-platform Restaurant Review Platform that allows users to explore restaurants, submit reviews, and upload photos through both web and mobile interfaces. The solution demonstrates a cloud-native, serverless, and secure architecture built on Amazon Web Services (AWS), using a modern TypeScript-based technology stack. The design emphasizes scalability, maintainability, and adherence to current industry and academic standards.

2. Objectives

1. Develop a complete full-stack system supporting both web and mobile platforms.
2. Implement secure authentication and role-based authorization using AWS Cognito.
3. Provide concurrent access to remote data hosted in Amazon RDS (PostgreSQL).
4. Enable secure media upload and delivery through Amazon S3 and CloudFront.
5. Track user last activity and device channel for analytic and behavioral purposes.
6. Deliver a solution that is scalable, cost-efficient, and built according to modern best practices in web and mobile development.

3. System Architecture

The system follows a layered, serverless architecture comprising web and mobile clients, a secure backend API, and a managed data layer. The layers include:

Client Layer	React Web Application and React Native (Expo) Mobile Application.
Security Layer	AWS Cognito User Pool for registration, authentication, and JWT management.
Application Layer	API Gateway + AWS Lambda (TypeScript) for stateless API logic.

Data Layer	Amazon RDS (PostgreSQL) and Amazon S3 for structured and unstructured data.
Delivery Layer	CloudFront and Route 53 for content distribution and DNS.
CI/CD Layer	GitHub Actions and Expo EAS for deployment automation.

4. Technology Stack and Rationale

Frontend	React (Vite, TypeScript).
Mobile	React Native with Expo for cross-platform capability.
Authentication	AWS Cognito with self sign-in and JWT validation.
Backend	AWS Lambda (TypeScript) deployed via Serverless Framework v3.
Database	Amazon RDS (PostgreSQL).
Media Storage	Amazon S3 with CloudFront and OAC.
CI/CD	GitHub Actions (OIDC) and Expo EAS.
Monitoring	Amazon CloudWatch for metrics and logs.

5. Functional Requirements

1. Multi-role access: Admin, Registered User, Guest User.
2. Authentication and registration using Cognito.
3. Admin-only access to management features.
4. Restaurant catalog filtering by city, category, tags.
5. Review submission, editing, deletion for registered users.
6. Image uploads via pre-signed URLs.
7. Tracking of user last login activity.
8. Public (guest) browsing with restrictions.
9. Consistent responsive UI across devices.
10. Accessibility compliance.

6. Data Model Design

Database entities:

- users: id, email, display_name, last_login_at, last_login_channel
- restaurants: id, name, city, categories, tags, cover_image_key
- reviews: id, restaurant_id, user_id, rating, comment
- review_images: id, review_id, s3_key, width, height, size_bytes

7. API Design

Catalog Service:

- GET /catalog/restaurants
- GET /catalog/restaurants/:id
- POST /catalog/restaurants (admin)
- PATCH /catalog/restaurants/:id (admin)
- POST /catalog/user-activity (user)

Review Service:
- POST /reviews/images/presign
- POST /reviews
- GET /reviews?restaurantId=
- DELETE /reviews/:id

8. Web Application

The web app is built using React and TypeScript, following responsive design and accessibility principles. It connects to backend APIs through HTTPS and uses React Query for caching and Axios for API requests. Light/dark themes and navigation consistency are maintained.

9. Mobile Application

The mobile app is built with React Native (Expo). It supports Cognito-based authentication, user activity tracking, camera and gallery integration, and offline queueing for uploads. OTA updates are delivered via Expo EAS Update.

10. Security and Compliance

- Cognito JWT validation via API Gateway Authorizer.
- S3 private bucket with CloudFront OAC.
- IAM roles with least privilege.
- Input validation and API throttling.
- HTTPS enforced across all endpoints.
- Secrets stored in AWS SSM Parameter Store.

11. Performance and Optimization

Optimizations include pooled PostgreSQL connections, lightweight Lambda bundles, CloudFront caching, S3 lifecycle rules, and regional hosting in ap-southeast-1.

12. Test Plan

Functional	Verify registration, login, CRUD operations.
Usability	Evaluate UI flow and accessibility.
Device	Test on multiple screen sizes and camera use.
Performance	API latency under concurrent load.
Security	Verify access restrictions.
Integration	Validate web/mobile/API coherence.

13. Marketing Plan

Pre-launch	Branding, domain registration, and social setup. (codelabs.lk)
Launch	CloudFront web deployment, EAS mobile builds, influencer engagement.
Post-launch	Gather user feedback and analytics.
Growth	Implement SEO, targeted advertising, and restaurant partnerships.

14. Publishing Plan

Web	Hosted via AWS CloudFront and S3 with HTTPS.
Backend	Deployed via GitHub Actions and Serverless Framework.
Mobile	Built using Expo EAS and prepared for Play Store and App Store publishing (demo phase currently).

15. Intellectual Property Policy

All project assets are owned by the team members:
- Krishantha Dinesh
- Denuwan Bandara
- Dilshika Bandara

All open-source software is used under proper licenses. No external organization holds rights to this project.

16. Project Plan and Roles

Team Roles:
Krishantha Dinesh – Architect, Backend Developer, DevOps (CI/CD)
Denuwan Bandara – Frontend Web Developer and Integration
Dilshika Bandara – React Native Mobile Developer and web UI

Timeline (1 Month):
Week 1 – Requirements, Cognito, and architecture setup.
Week 2 – Backend and database development.
Week 3 – Web and mobile application development.
Week 4 – Testing, deployment, and demo preparation.

Demonstration will showcase: authentication, role-based access, media uploads, user activity tracking, and data synchronization between platforms.

17. Evaluation and Reflection

The project successfully combines serverless backend design, cloud-native storage, and cross-platform frontends. It reinforced practical knowledge in AWS architecture, TypeScript development, and continuous delivery practices.

18. Conclusion

The Restaurant Review Platform achieves the objectives of secure, scalable, and user-friendly web and mobile application development. The AWS ecosystem ensures reliability and performance, while the React and Expo frameworks enable a seamless cross-platform experience.